Sistemas Distribuidos I



Entrega IIII: Tolerancia Fallas

Anexo

PADRÓN	ALUMNO	MAIL
104414	Correa, Valentina Laura	vcorrea@fi.uba.ar
104046	Andrade, Agustin Ariel	aandrade@fi.uba.ar
101231	Dimartino, Pablo Salvador	pdimartino@fi.uba.ar

Grupo: 20

Cuatrimestre: 1c 2025

Introducción

El presente es un anexo que busca explicar cuales fueron los errores detectados previos a la demostración de esta entrega. Asi mismo, a continuación se detalla la manera en la que fueron resueltos.

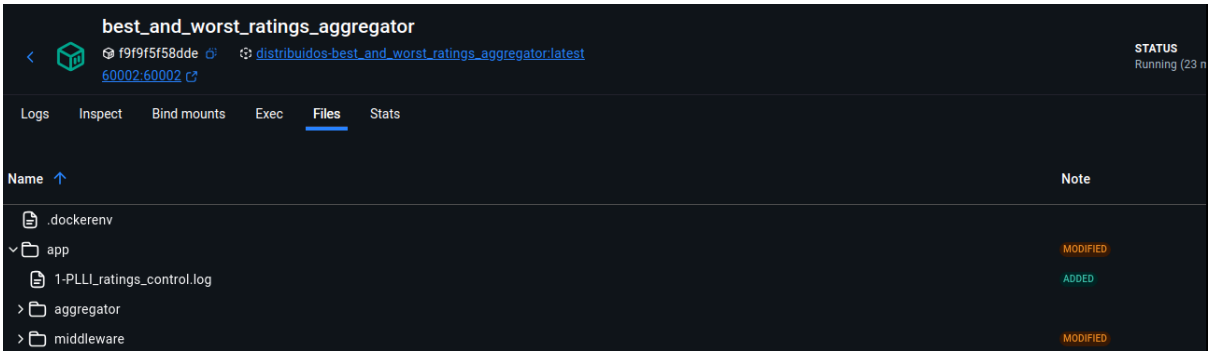
Problemas detectados

Limpieza de archivos

Problema

Pese a que se tuvo en cuenta la funcionalidad de limpieza de recursos utilizados luego de las caidas de los nodos, lo que se detecto fue que no se estaban eliminando los archivos de log (creados para el recupero de mensajes) ni en los joiners ni en sus respectivos aggregators. El escenario en donde ocurria era luego de matar el container de un cliente a la mitad del envio de sus archivos.

La siguiente captura muestra el resultado de una caida luego de matar un cliente durante el envio del archivo de ratings.



Detección

En nuestro trabajo si se tuvo en cuenta la eliminación de recursos. La manera en la que lo resolvimos fue a través del uso de mensajes envenenados. Lo que hace esto es fingir un mensaje con el acumulado actual enviado fingiendo una petición correcta y luego descartando los resultados obtenidos.

Si bien esta lógica era correcta, en los joiners no se estaba handleando el flag correspondiente (is_final) perteneciente a la identificación de si era un mensaje de este tipo o no.

Solución

Se resuelve agregar en los joiners el control de este tipo de mensajes en el metodo handle_control_message. Aqui lo que se hace es remover los archivos existentes, eliminacion de datos y todas las limpiezas necesarias. Ademas de esto, se le envia el mensaje a su respectivo aggregator. Finalmente, se le hace un ack de ese mensaje a rabbitmq para darle a entender que ya fue procesado y no se debe encolar nuevamente.

Validación

Se incluye una demo en el video adjunto.

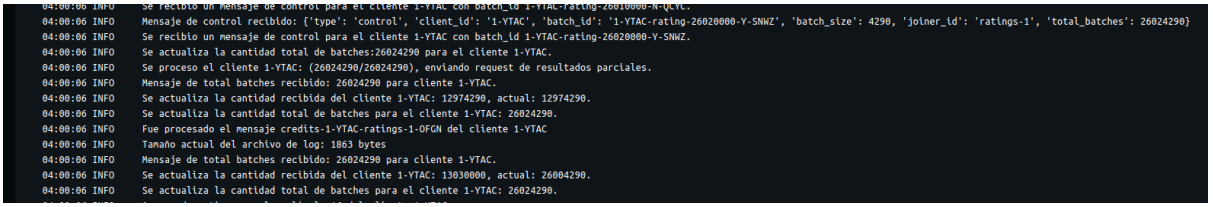
No envío de respuesta en aggregators de joiners aleatoriamente

Problema

Dado un cierto numero de procesamientos completos del programa, ocurría que en algunas ocasiones aleatoriamente el aggregator de los joiners recibia las respuestas de los mensajes procesados de estos, pero entre si no alcanzaban a hacer la sumatoria total de los mismos. Dado que esto no ocurría de manera consistente resulto un poco mas compleja su detección ya que previamente se consideraron distintas teorías como estar en presencia de condiciones de carrera y demas. Finalmente notamos que estas no eran la causa raiz.

Detección

Desde el log del aggregator se podía visualizar el siguiente log:



Repasando nuestro flujo en la aplicación, una vez que el aggregator recibe un registro que contiene el numero total de batches del archivo (solo presente en el último), envia una consulta a todos sus aggregators para que le envien la cantidad de batches que proceso cada uno de ellos, y asi validar que efectivamente se habia procesado la totalidad del archivo.

Observando el log, se puede notar que el último batch enviado registra una cantidad total de **26024290** batches. Luego se realizan las peticiones a cada uno de los joiners, en donde los mismos responden que cada uno proceso cantidades de **12974290** y **13030000**, las cuales sumadas resultan en un total de **26004290**, dando como resultado una perdida de **20000** batches.

Al ser esta cantidad diferente a la total, el aggregator nunca entraba en la condición de procesamiento finalizado lo cual resultaba en el no envío del resultado final al cliente.

Luego de hacer un repaso minucioso de por donde iba el flujo en nuestro código, pudimos finalmente notar algunos errores pequeños que en conjunto desencadenaban en este resultado.

Orden de operaciones

Por un lado, se realizaba un orden incorrecto en dos operaciones, las cuales se invocaban en la inicializacion de los aggregators.

Inicialmente nosotros teniamos las operaciones de levantar el server y realizar un recover de mensajes perdidos, en ese orden. Recordando nuestro flujo, cada vez que al joiner le llega un mensaje de credits, este antes de procesarlo lo que hace es ir a consultar por tcp al aggregator si puede procesarlo o hubo alguien que ya se encargo de hacerlo.

Luego de la caída de todos los nodos a la vez, lo que sucedia era que en los escenarios en donde el server ya habia inicializado pero aun no se habia producido el recover de mensajes, al joiner le llegaban nuevas requests de mensajes de credits. En cada una de estas, al no tener los datos correctamente cargados aun, daba respuestas erroneas.

Guardado de batch id

Adicionalmente, notamos que unicamente en el aggregator de ratings no se estaba realizando adecuadamente el registro de asociacion de un batch_id a su respectivo joiner; es decir no se estaba cargando ese valor en el diccionario batches ids - joiners ids adecuadamente.

Configuración en colas

Agregamos la definicion de nombres a las colas. Sumado a esto, en el publisher teniamos la linea:

self.channel.exchange_declare(exchange=exchange_name, exchange_type='fanout')

la cual no especifica durabilidad. Investigando esta función notamos que el valor por default de durable es False. Corregimos esto agregando explicitamente ese tag y seteandolo en True.

Además, el nombre de las colas exclusivas a las que se conectaba un consumidor, era un nombre aleatorio, con lo cual cada vez que se levantaba se conectaba a una cola nueva. Es por esto que no se podia contemplar lo que sucedia previo a esto.

Solución

Se invierte el orden de las operaciones de inicio del server tcp de los aggregators y el repero de transacciones. Se aplican los cambios de configuración a las colas. Se realiza guardado correcto de batchs ids y joiner ids.

Validación

Se incluye una demo en el video adjunto.